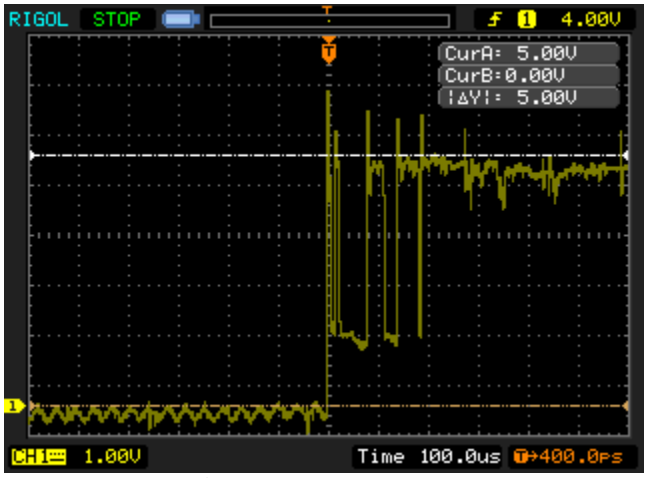
Debouncing in VHDL

Problem

When a mechanical switch or button is pressed, the state of input is not always changed just once. Instead, switch/button may "bounce," causing contacts to open/close multiple times.



Signal input from switch being turned ON (100us/div)

Solution (VHDL)

Step 1: Store current and last state of input

Step 2: Based on clock frequency (used to sample input) and required period to consider input "stable," we can calculate the required number of consistent (same state) samples needed to consider the input "stable."

Step 3: Once the input is stable, pass the state of the input into the rest of the VHDL.

Code (.vhd file on Canvas)

```
-- Switch/button debouncing VHDL entity

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY debounce IS
  GENERIC(
    clk_freq    : INTEGER := 50_000_000;  --system clock frequency in Hz
    stable_time : INTEGER := 10);         --time button must remain stable in ms
  PORT(
    clk     : IN  STD_LOGIC;  --input clock
    button  : IN  STD_LOGIC;  --input signal to be debounced
    result  : OUT STD_LOGIC); --debounced signal
END debounce;

ARCHITECTURE logic OF debounce IS
  SIGNAL flipflops   : STD_LOGIC_VECTOR(1 DOWNTO 0); --input flip flops
  SIGNAL counter_set : STD_LOGIC;                    --sync reset to zero
BEGIN

  counter_set <= flipflops(0) xor flipflops(1);  --determine when to start/reset counter

  PROCESS(clk)
    VARIABLE count :  INTEGER RANGE 0 TO clk_freq*stable_time/1000;  --counter for timing
  BEGIN
    IF(clk'EVENT and clk = '1') THEN                    --rising clock edge
      flipflops(0) <= button;                           --store button value in 1st flipflop
      flipflops(1) <= flipflops(0);                     --store 1st flipflop value in 2nd flipflop
      If(counter_set = '1') THEN                        --reset counter because input is changing
        count := 0;                                     --clear the counter
      ELSIF(count < clk_freq*stable_time/1000) THEN     --stable input time is not yet met
        count := count + 1;                             --increment counter
      ELSE                                              --stable input time is met
        result <= flipflops(1);                         --output the stable value
      END IF;
    END IF;
  END PROCESS;

END logic;
```

Adding “debounce” to a VHDL Project

1. Add the file
    a. Project → Add/Remove Files in Project
    b. Navigate to *debounce.vhd*, select, and add file to project

2. Add the debounce component
    a. Recall: COMPONENT should match debounce entity PORT

```
component debounce is
        PORT( clk : IN  STD_LOGIC;  --input clock
              button : IN  STD_LOGIC;  --input signal to be debounced
              result : OUT STD_LOGIC); --debounced signal
END component;
```

3. Debounce inputs using debounce component
    a. Recall: PORT MAP – order of arguments must match COMPONENT and entity PORT

```
-- Debounce CLK key input - clk = 50MHz system clock
-- Clock <= KEY(0);
CLKDB: debounce PORT MAP (CLOCK_50, KEY(0), Clock);
```

Or

```
CLKDB: debounce port map (CLK_50, CLK, OUTCLK);
```

The signal name depends on your own definition

    b. For clock input, use 50MHz system clock
        i. Declare as input in top-level entity port

```
ENTITY part1 IS
   PORT (

            CLOCK_50   : IN  STD_LOGIC; --System clock

END part1;
```

    c. In the Pin Assignment Editor, assign input CLOCK_50 to PIN_P11.

4	✔	[redacted]	CLOCK_50	Location	PIN_P11	Yes